Hands-On Machine Learning

Chapter 4 — Training Models

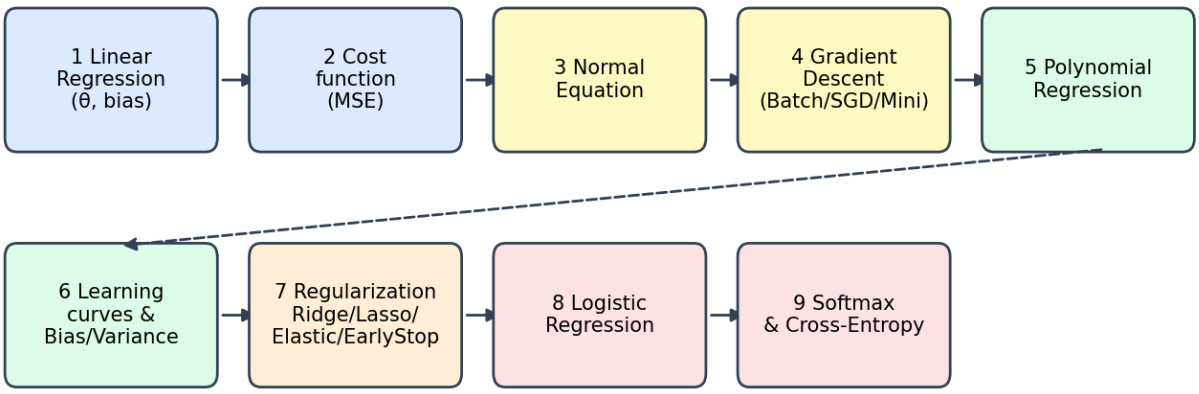
Linear · Polynomial · Regularized · Logistic · Softmax

Summary of your notebook + the missing points filled in + figures and code for revision

From your notebook **Added to fill gaps** **Note / correction**

The numbers in the green boxes came from actually running the code. The figures marked (synthetic) or (illustrative) use generated data for illustration only. The Iris figures are from the real data.

Chapter map: each step solves the problem of the step before it.



1 Linear Regression: the idea and the Parameters

From your notebook

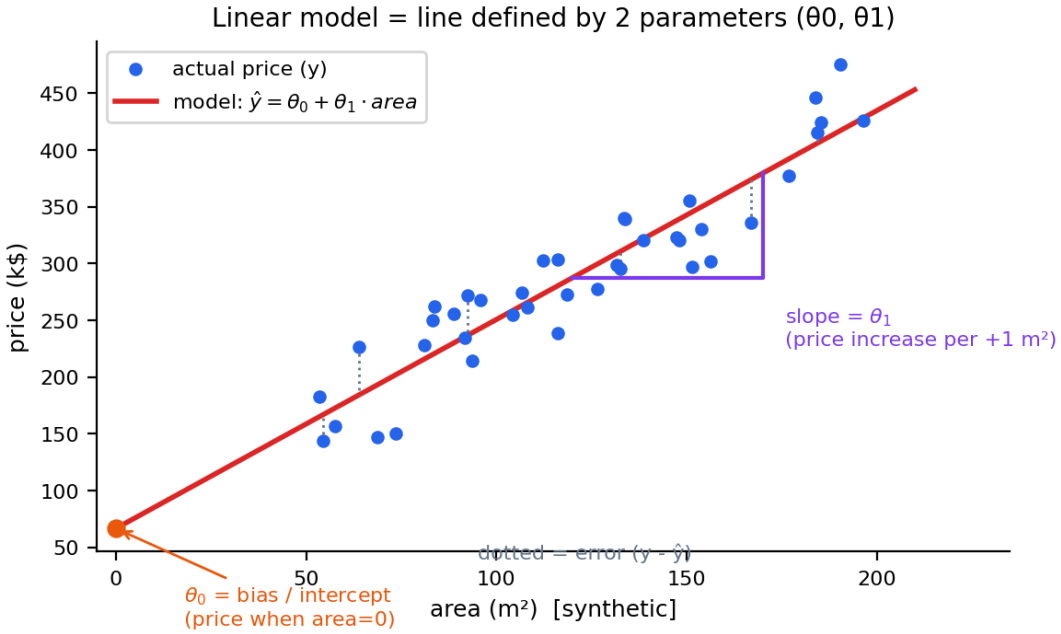
Linear Regression: the idea is simple: I have `features` and I want to predict a numeric value. ← `X -> Model -> ŷ`

The linear model makes the prediction by: **Weighted sum of Features + Bias**.

- `ŷ` = the value the model predicted.
- Features = x1, x2, ..., xn
- weights / parameters = θ1, θ2, ..., θn
- Bias / Intercept = θ0 (the bias / the constant term)

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$\hat{y} = h_\theta(\mathbf{x}) = \theta \cdot \mathbf{x}$$



A linear model = a line defined by 2 parameters. θ0 is the point where the line crosses the y axis, and θ1 is the slope of the line. The dotted lines = the error.

From your notebook

Example: if I have `Price = θ0 + θ1 × area + θ2 × rooms`, the model learns the values `θ0, θ1, θ2` so that the predictions are close to the real values.

From your notebook

Parameters: these are the values the model itself learns from the data. For example: `θ0 = 100`, `θ1 = 20`, `θ2 = 500`.

When the `Linear Regression` trains it tries to find the best values for `θ` so that the error is minimized. ← **Training = finding the best values for the Parameters.**

And these are different from the `Hyperparameters` which we were dealing with in Chapter 2: the Hyperparameters **we** set, but the Parameters **the model** learns.

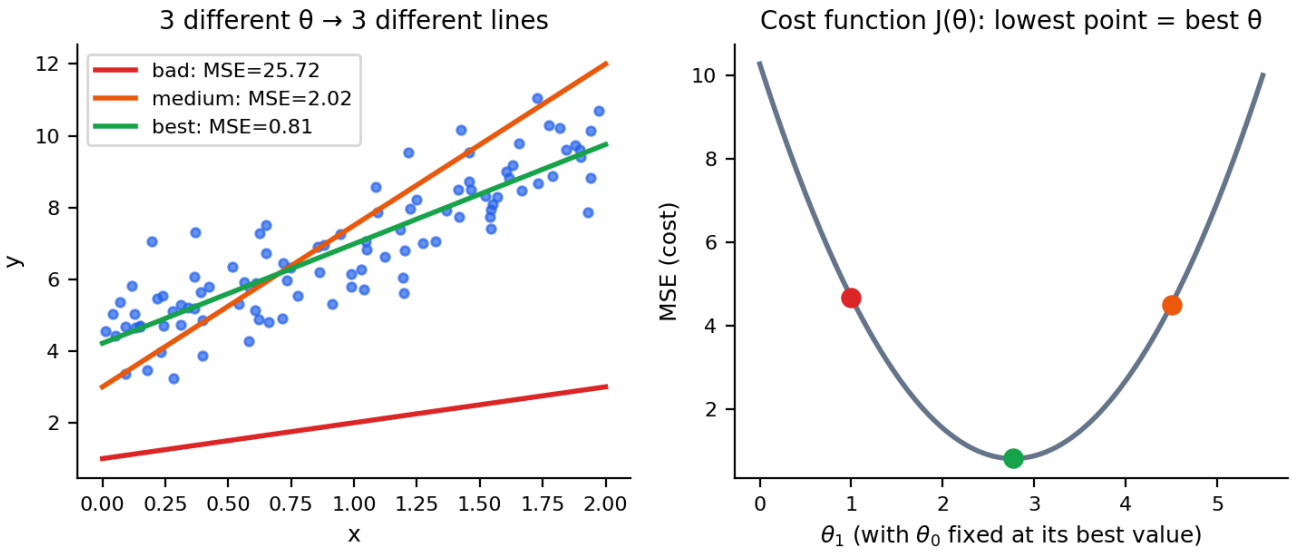
Addition

With the numbers in your example: if a house has `area = 100` and its number of rooms `rooms = 3`: `ŷ = 100 + 20×100 + 500×3 = 3600`. Every time the θ change, the prediction changes; that is why training = searching for θ.

The Bias trick: we add a column of all 1s to the features (`x0 = 1`) so we can write the whole equation as a vector multiplied by a vector: `ŷ = θ · x`. And so the matrix `X` becomes of shape `(m, n+1)` (number of samples × (number of features + 1)).

	Parameter	**Hyperparameter**
Who sets it?	the model learns it from the data	we set it before training
Examples	θ0, θ1, ... (weights)	learning rate η, epochs, alpha, degree, number of epochs
Does it change during training?	Yes	No (unless we did the tuning ourselves)

2 The Cost Function and the difference between Loss and Metric



Left: 3 different values of θ = 3 different lines, each with its MSE. Right: the cost as a function of θ1: the lowest point is the best θ. (Numbers from a real run.)

From your notebook

We know that the `linear regression` makes a `prediction` using `θ`, but who will choose `θ`? We must choose values that make the model the best it can be on the `training set`, and to know what is "best" we must use a `Cost function`.

From your notebook

Cost Function: any function that measures "how bad the model is" based on certain `theta` values: like `MSE`, `RMSE` (it takes `θ` as input and returns one number representing "the degree of bad performance").

`MSE` itself (as a concept) is a general evaluation criterion, but when we use it as a function that takes `θ` and tells you "how large the error is with exactly these values" it becomes the `Cost function` that we are trying to minimize.

$$\text{MSE}(\theta) = \frac{1}{m}\sum_{i=1}^{m}(\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$$

From your notebook

An important point: `Training loss ≠ Evaluation metric` necessarily. The algorithm may use a certain `loss function` during training, and after training we measure with a different `metric`.

The reason is that the `loss` may be easier computationally to optimize, or may have extra components specific to the training process (like `Regularization`).

← What matters is that the `Training objective` must be strongly related to the final goal of the project.

3 The Normal Equation

```
import numpy as np
from sklearn.preprocessing import add_dummy_feature
np.random.seed(42)
m = 100
X = 2 * np.random.rand(m, 1)
y = 4 + 3 * X + np.random.randn(m, 1)      # 4 + 3x + Gaussian noise
X_b = add_dummy_feature(X)                 # adds x0 = 1 to every instance
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
print(theta_best)
```

Addition

An example from the book (we reach it at the end of the chapter): Logistic Regression trains with `log loss` but afterwards we evaluate it with `Precision / Recall`. The two are different, and that is normal. And also `MSE` and `RMSE` reach the same best `θ` (because the square root is an always-increasing function), but `MSE` is easier in the derivative, so it is the one we use in training.

Note / Correction

Remember the correction of Chapter 2: if you want `RMSE` in recent versions of scikit-learn use `root_mean_squared_error(y_true, y_pred)` instead of `mean_squared_error(..., squared=False)`, which was removed.

From your notebook

For `linear regression` there is a mathematical way that gets you the best `Parameters` **directly** without doing `iterations`: **Normal equation**:

The meaning: use the data itself in a mathematical equation so that the values of the `Parameters` come out which minimize the `MSE`.

$$\hat{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

From your notebook

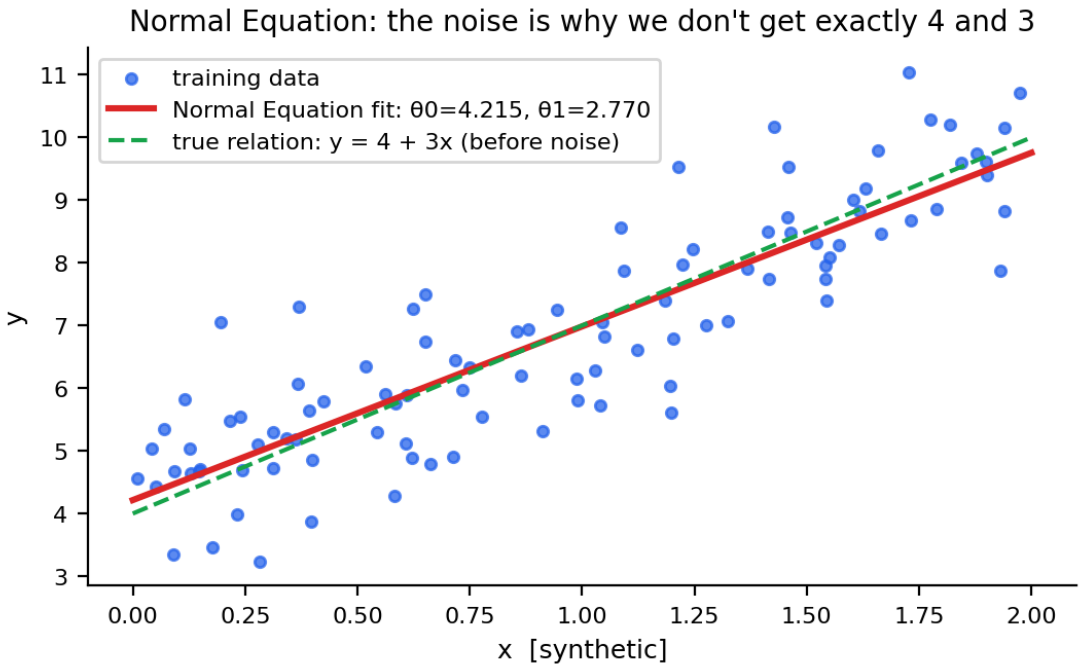
The book's code (Normal equation): the first thing we generate approximately linear data: `y = 4 + 3X`, but the book added `Gaussian noise`. Therefore when the model tries to recover the relation it does not necessarily come out exactly 4 and 3.

After that we add the `Bias column`: `add_dummy_feature(X)`: this adds `x0 = 1` to every `instance`. After that we can compute the `bias` inside the `matrix operation`.

Output (real run)

```
[[4.2151]
 [2.7701]]
```

```
X_new = np.array([[0], [2]])
X_new_b = add_dummy_feature(X_new)        # add the bias column to the NEW data too
y_predict = X_new_b @ theta_best          # y_hat = X . theta
print(y_predict)
```



The result of the Normal Equation on this data: the red line is very close to the real relation (green) but not identical, because of the noise.

From your notebook

This is a direct implementation of the `Normal equation`, and `@` is `Matrix multiplication` in numpy. The result came out `θ0 = 4.215` and `θ1 = 2.770`, not exactly 4 and 3 because of the `noise`.

The book used `θ` after we obtained it for the `prediction`:

Output (real run)

```
[[4.2151]
 [9.7553]]
```

From your notebook

That is: take the new `Features`, add the `bias column`, multiply them by the `Parameters` (`θ`), and get the `prediction`. And this is the same concept `ŷ = Xθ`.

From your notebook

So why do we do all this manually? To understand what happens under the `hood`, but in reality you will use:

```
from sklearn.linear_model import LinearRegression
lin_reg = LinearRegression()
lin_reg.fit(X, y)
print(lin_reg.intercept_, lin_reg.coef_)       # theta0 , theta1 ...
print(lin_reg.predict(X_new))

theta_svd = np.linalg.pinv(X_b) @ y            # same answer through the pseudoinverse
print(theta_svd)
```

Output (real run)

```
[4.2151] [[2.7701]]
[[4.2151]
 [9.7553]]
[[4.2151]
 [2.7701]]
```

From your notebook

After training: `lin_reg.intercept_` ← represents `θ0`, and `lin_reg.coef_` ← `θ1, θ2, ...`.

From your notebook

What does the linear regression itself use? The book says it is built on `scipy.linalg.lstsq()` which means `Least squares`. And it uses a method based on the `pseudoinverse` instead of executing the `normal equation` directly, because `X^T X` may be non-invertible, but the `pseudoinverse` works even if it is non-invertible.

← **Scikit-learn:** you do not need to write the `Normal equation` yourself, it is what takes care of the calculation in a more stable way (`stability`).

From your notebook

Why don't we always use the normal equation? The `normal equation` needs the computation of `(X^T X)^-1`, and as the number of `features` `n` grows, the `Matrix inversion` operation becomes computationally more expensive. And the book mentions that the cost may be `O(n^2.4)` to `O(n^3)` depending on the implementation. Likewise if the number of `features` is very large these methods become slow.

Addition

When is `X^T X` not invertible? When there are `features` that are duplicated or perfectly correlated (for example `area_m2` and `area_ft2` together), or when the number of samples `m` is less than the number of features `n`. Then `inv` fails or gives an unstable result, and the `pseudoinverse` (`pinv`) is the solution.

A nice point: the Normal Equation is **linear** with the number of samples `m` (i.e. 10× the data = roughly 10× the time), so its real problem is in the growth of `n`, not `m`. And after training the prediction itself is very fast (linear in `m` and `n`).

4 Gradient Descent

From your notebook

`Gradient Descent` is an `Optimization Algorithm`. Its idea: it adjusts the `Parameters` gradually until it minimizes the `Cost function`.

Imagine you are on top of a mountain in fog and cannot see the valley, but you can feel the slope of the ground, so you go down in the direction of the descent ← this is exactly the book's analogy.

In `ML` : `Cost function` ← `Gradient` ← I know the direction of error increase ← I move in the **opposite** direction (so the error decreases).

From your notebook

What is the Gradient? It is the set of `partial derivatives` with respect to each `Parameter`. That is, instead of asking: if I changed `θ1` a bit, how much would the `cost` change? We ask this question for every `Parameter`: `θ0 -> derivative`, `θ1 -> derivative`, ..., `θn -> derivative`. And we gather them in a `Gradient vector`.

$$\nabla_\theta \text{MSE}(\theta) = \left(\frac{\partial \text{MSE}}{\partial \theta_0}, \frac{\partial \text{MSE}}{\partial \theta_1}, \ \ldots, \ \frac{\partial \text{MSE}}{\partial \theta_n} \right)$$

From your notebook

Important: the `Gradient Descent` equation: this is the heart of `GD`: read it like this: **new value = current value − the descent step**. The step depends on: `η` (`learning rate`) and the `Gradient` (= direction and amount of change of the `cost`).

$$\theta^{(\text{next step})} = \theta - \eta \nabla_\theta \text{MSE}(\theta)$$

Addition

The `Gradient vector` of the `MSE` in Linear Regression has a ready formula (we will use it in the code coming):

$$\nabla_\theta \text{MSE}(\theta) = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y})$$

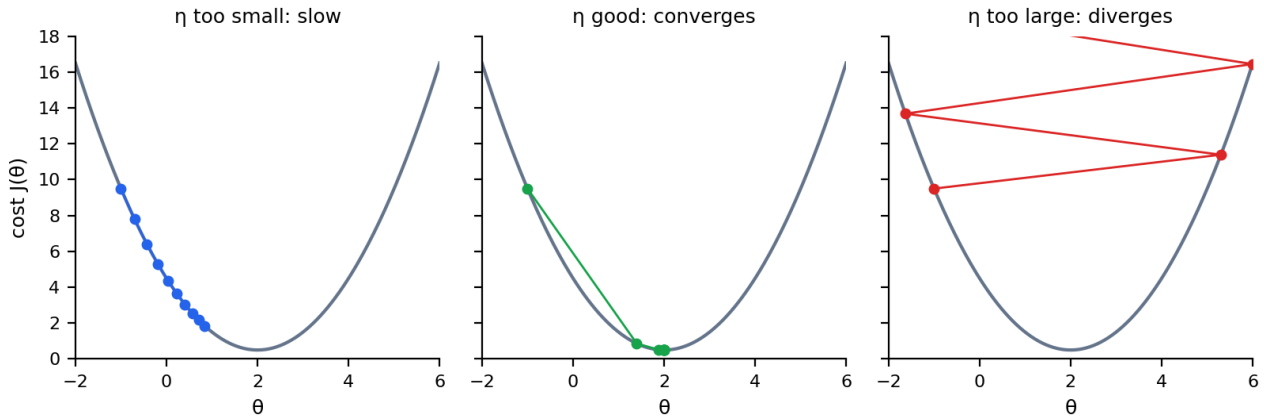
Addition

Notice that the equation uses **all the data** (`X` complete) ← this is exactly the name **Batch Gradient Descent** which comes shortly after.

From your notebook

What does the Learning Rate mean? It is the size of the step with which you move in each `Iteration`:

- If **very small**: it reaches the `minimum` but very slowly (slow convergence).
- If **large**: it may jump over the `minimum`, may not settle, and may go further away.

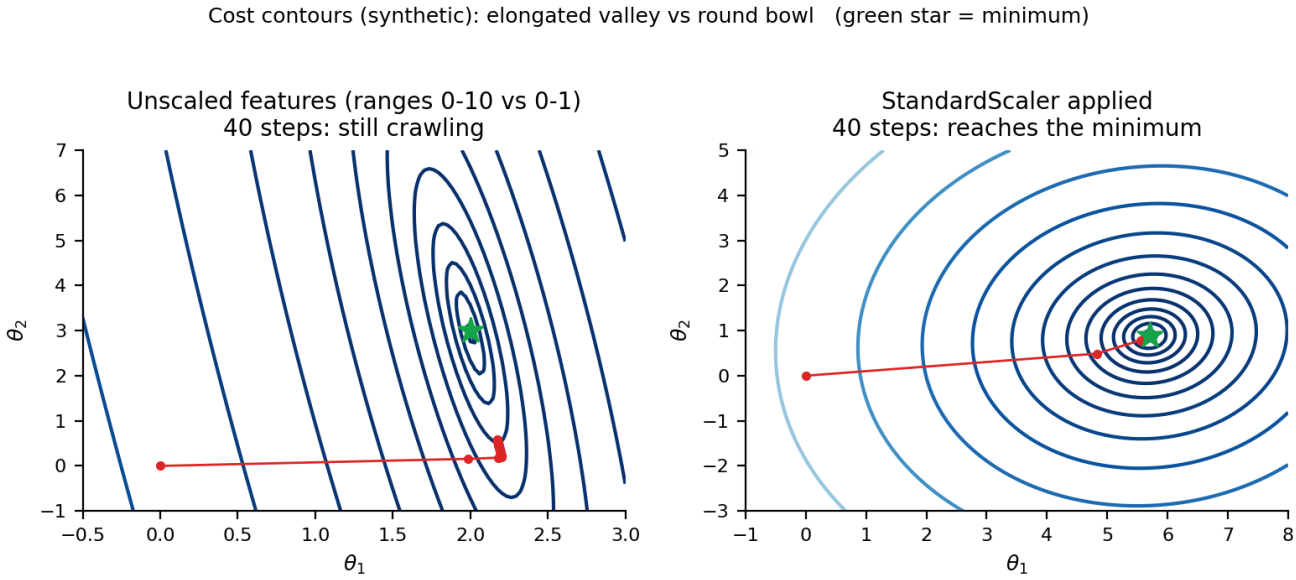


The same function and the same starting point with 3 values of η: a small one crawls, a suitable one reaches in few steps, and a large one jumps far away and moves away from the solution.

From your notebook

Why is Scaling very important with Gradient Descent? If I have `features` of very different sizes, the shape of the `Cost surface` becomes like **a long thin valley** instead of being closer to a regular bowl, so the `GD` moves indirectly and needs a longer time to reach the `minimum`.

Therefore with `GD` it is important that the `features` are on close `scales`, for example using `StandardScaler`.



The same number of steps (40) and the same learning rate: without scaling the red line is still crawling in the valley, and with StandardScaler it reached the green star. (synthetic data)

Addition

The Normal Equation does **not** need scaling, but anything that uses Gradient Descent (in its types + `SGDRegressor` + the Regularized models) **needs** it.

When do I stop? The book says: make the number of `epochs` large, and stop when the `Gradient vector` becomes very small (smaller than a small value called `ε` `tolerance`), because then you have roughly reached the `minimum`.

5 Batch Gradient Descent

```
eta = 0.1                               # learning rate
n_epochs = 1000
theta = np.random.randn(2, 1)           # random initialization

for epoch in range(n_epochs):
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)
    theta = theta - eta * gradients
print(theta)
```

Output (real run)

```
[[4.2151]
 [2.7701]]
```

From your notebook

The Convex Function: the book talks about the `Convex function`: the `MSE` `Cost function` in `linear regression` is `convex`, and this is very important because it means that we have **one** `Global Minimum` and there are no `local minima`.

So if we used `Gradient Descent` appropriately it is **guaranteed** to approach the global solution as long as training continues long enough and the `learning rate` is suitable. ← That is why `GD` is suitable for `linear regression`.

Addition

The definition of `convex`: simply: if you draw a straight line between any two points on the curve, this line never goes **below** the curve. Its shape is a bowl: there are no side pits to get stuck in. (The cost functions of Neural Networks are **not** `convex`, and there the `local minima` and plateaus problem is real. Only Linear and Logistic Regression are lucky.)

From your notebook

In every `step`: use the **whole** `Training set` to compute the `Gradient`. That is: the whole `Training set` ← compute `Gradient` ← update `θ`. And this makes every `step` expensive if the data is huge.

What is an `epoch`? One complete pass over the `Training set`. In `Batch GD` each `epoch` means you compute the `Gradient` using all the data once.

From your notebook

The idea: `θ` random at the start, compute the `Gradient`, update `θ`, repeat, and approach the best `θ`. And at the end it reached the same result that the `Normal Equation` gave, approximately. And this is an important thing: two different training methods can reach the same solution.

6 Stochastic Gradient Descent (SGD)

```
n_epochs = 50
t0, t1 = 5, 50        # learning schedule hyperparameters (control the shape of eta)

def learning_schedule(t):
    return t0 / (t + t1)     # eta depends on the current training step number t
```

Addition

Compare the Output with `theta_best` in section 3: roughly the same numbers (4.215 and 2.770). And an experiment on the same code (real results after 1000 epochs): `eta=0.02` reached 4.2144, 2.7707 (very close but a bit slower), but `eta=0.5` blew up (numbers on the order of 10^27), and `eta=1.1` gave `nan`. This is the divergence we saw in the previous figure.

From your notebook

Here the book solves the problem of `Batch GD`: **slow** because it uses all the data in every `step`.

In `Stochastic Gradient Descent`: every `step` picks one `instance` randomly and computes the `Gradient` based on it only.

The advantage: every `step` is much faster, and it can deal with very huge `Datasets`.

The disadvantage: the path becomes `Noisy / Stochastic`: the `Cost` may go up and down instead of only decreasing.

From your notebook

The Learning Schedule in SGD

Here `SGD` : the `Randomness` means it keeps moving around the `minimum` instead of settling on it.

The solution: reduce the `learning rate` gradually. At the start a large `learning rate` ← progress quickly, then a smaller `learning rate` ← we approach the `minimum` . And the function that determines the `learning rate` value at each `step` is called the `Learning Schedule` .

$$\eta(t) = \frac{t_0}{t + t_1}$$

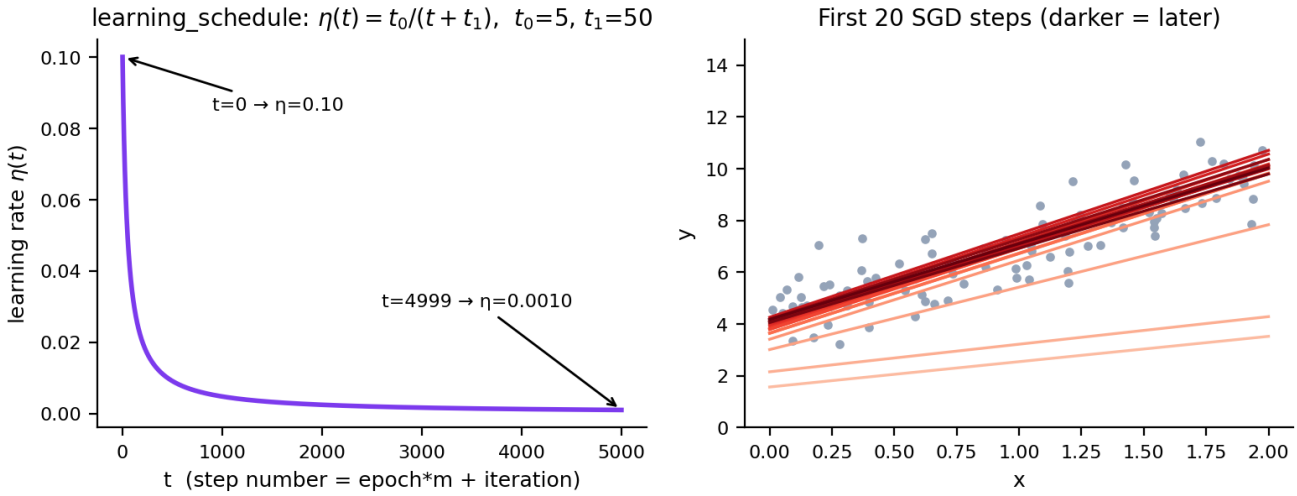
From your notebook

These are two values controlling the shape of the `learning rate` . And the function computes the `learning rate` based on the number of training steps (the current `t`).

From your notebook

Let's understand what problem we are solving in the first place: you know that `Gradient descent` updates the `Parameters` with `θ_new = θ_old − η ∇θ MSE` . Here `η` is the `learning rate` , i.e. the size of the step with which the `Parameters` move.

The problem: you do not necessarily want the step size to stay constant for the whole training. At the start we need relatively large steps to approach the solution quickly, and after that we need smaller steps so as not to keep jumping around the solution. This is the role of `learning_schedule()` . Note: whenever the value of `t` increases the `learning rate` decreases.



Left: the learning rate decreases from 0.10 to about 0.001. Right: the first 20 steps of SGD (from a real run): the light lines started far and the dark ones are close to the solution.

```
np.random.seed(42)
theta = np.random.randn(2, 1)      # initialization of the parameters
```

```
for epoch in range(n_epochs):
    for iteration in range(m):
        random_index = np.random.randint(m)                # pick a random sample index 0..m-1
        xi = X_b[random_index : random_index + 1]          # its features
        yi = y[random_index : random_index + 1]            # its real target
        gradients = 2 * xi.T @ (xi @ theta - yi)           # gradient from ONE sample only
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients
print(theta)
```

Output (real run)

```
[[4.2108]
 [2.7486]]
```

In short: the `learning schedule` is a function that determines how the `learning rate` changes during training instead of staying constant.

From your notebook

In the shape `(2, 1)` because the example has **one** `feature` (but we added a `Bias` column using `X_b = add_dummy_feature(X)`). So we now have two coefficients required to learn: `θ0` ← `Bias / Intercept`, and `θ1` ← the weight of the (only) `feature`. We started with random values, and after that `SGD` will try to improve them gradually.

From your notebook

The main training loop (the code): in the previous example `m = 100`, so we do `m` steps inside each `epoch` (`m` = the number of training samples). That is:
`50 epochs × 100 steps = 5000 updates`.

From your notebook

- `random_index`: picks an `index` randomly between `0` and `m-1`. So if it picked sample 24 then `xi` = the `Features` of sample 24, and `yi` = the real value of the sample.
- `gradients`: computing the `Gradient` for **one sample** instead of computing the `Gradient` using the entire `dataset` (`Batch`). Here we compute the `prediction` (`xi @ theta`) and compare it with `yi`.
- `theta = theta - eta * gradients`: the same update of the `Parameters`: it means: take the current values of the `Parameters` and move them **in the opposite direction** of the `Gradient` by an amount determined by the `learning rate`.

From your notebook

Why must the sample be iid, or do we shuffle?

`IID` (short for `Independent and Identically Distributed`) i.e. the samples are **independent** of each other and taken from the **same** probability distribution.

Imagine your data is sorted by the `label`: all samples of class A, then B, then C. If you trained `SGD` in this order, your updates at the start would be affected by the samples of A, and after that it moves to B. This can make the training path **unbalanced** and affect the convergence.

Therefore we can `Shuffle` the data while moving the `Features` and the `Targets` together to preserve each sample matching its answer.

From your notebook

Using `SGDRegressor` instead of writing the code manually:

```
from sklearn.linear_model import SGDRegressor
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None,
                       eta0=0.01, n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel())
print(sgd_reg.intercept_, sgd_reg.coef_)
```

Output (real run)

```
[4.2128] [2.7727]
```

From your notebook

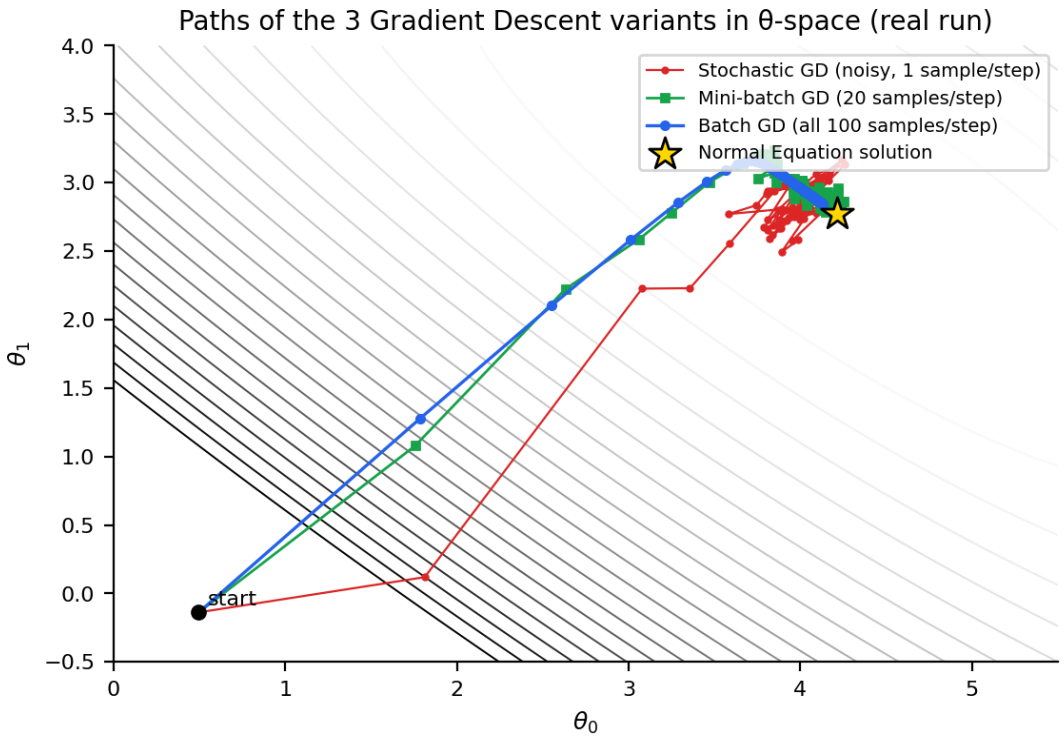
- `max_iter`: the maximum number of `epochs` (the maximum amount for continuing in training).
- `penalty=None`: not using `Regularization`.
- `eta0`: the initial `learning rate` value.
- `n_iter_no_change`: the number of consecutive `epochs` it monitors for lack of sufficient improvement before stopping training.

Addition

`tol=1e-5` means: if the `loss` did not go down by more than `0.00001` over `n_iter_no_change=100` consecutive epochs, training stops. And this is built-in **Early stopping** (we will talk about it in section 12).

Remark: `SGD` needs `feature scaling` like any `GD`, and in real data you must put `StandardScaler` before it in a `pipeline`.

7 Mini-Batch Gradient Descent



The paths of the three types in the (θ0, θ1) space from a real run: Batch (blue) smooth, Mini-batch (green) has a bit of oscillation, SGD (red) very noisy around the solution. The star = the solution of the Normal Equation.

Another advantage of randomness: if the `cost` is not `convex` (like Neural Networks) the oscillation of `SGD` may help it get out of `local minima`, but after that it needs a `learning schedule` to settle. The book likens it to `simulated annealing`.

From your notebook

A middle solution between `Batch` and `SGD`: we use **small groups** of `instances` (called `Mini-batches`) in each `step`.

Its advantage: achieves a balance between the stability of `Batch` and the speed of `SGD`; and among its advantages is that it benefits from `hardware` acceleration for `matrix` operations, especially with `GPUs`.

Addition

The complete comparison table between the training methods (from the book, adapted):

Algorithm	Large m (number of samples)	Large n (number of features)	Hyperparameters	Needs Scaling?
Normal Equation	fast (linear in m)	slow (about O(n^2.4..3))	0	No
SVD (LinearRegression)	fast	slow (about O(n^2))	0	No
Batch GD	slow	fast	2 (η and epochs)	Yes
Stochastic GD	fast (can work out-of-core)	fast	≥ 2 (+ schedule)	Yes
Mini-batch GD	fast	fast	≥ 2 (+ batch size)	Yes

All of them after training give roughly the same result in Linear Regression. The difference is in speed, memory, and the ability to deal with huge data.

8 Polynomial Regression

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)
lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)
print(X[0], X_poly[0])
print(lin_reg.intercept_, lin_reg.coef_)
```

Output (real run)

```
[-0.7528] [-0.7528  0.5666]
[1.7813] [[0.9337 0.5646]]
```

From your notebook

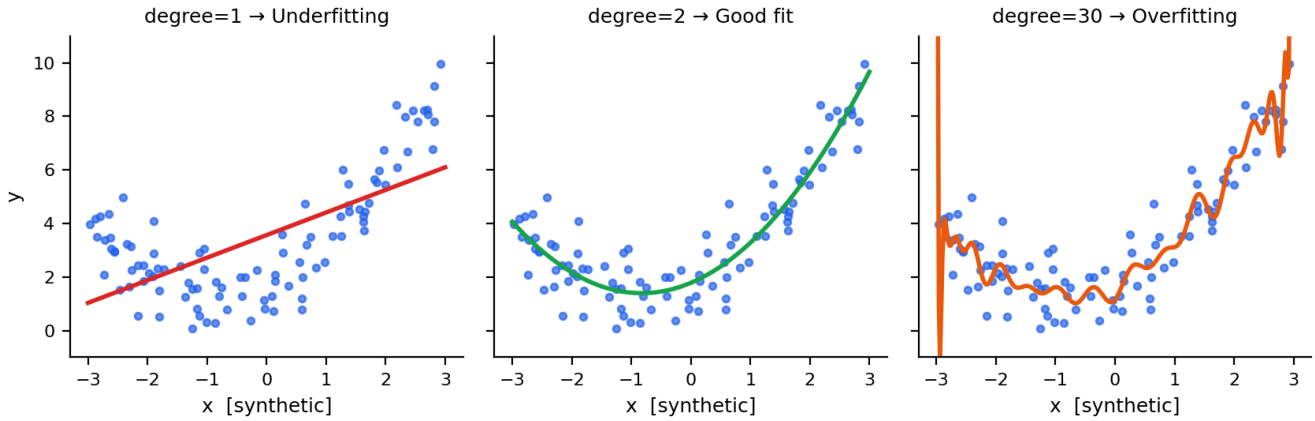
The problem: `Linear regression` assumes the relationship between the `features` and the `Target` is **linear**, i.e. roughly `y = ax + b`. But what if the data is curved? Like the real relation `y = ax² + bx + c`: here the straight line will not be able to represent the data well.

The book solves the problem by transforming the `features` before giving them to the `linear Regression`.

The idea: add new `Features` that are `Powers` of the original `Features`. Example: we have `x`, we convert it to `x` and `x²`, and then we train `linear regression`:

The model here is still `linear` in the `Parameters`, although the relationship with `x` itself became non-linear. ← `Polynomial Regression` = add polynomial features, then use `linear regression`.

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$$



The same data (synthetic, its real relationship is quadratic) with 3 degrees: degree 1 cannot catch the curve, degree 2 is right, degree 30 wraps around every point.

From your notebook

- `include_bias=False` : i.e. do not add the column of `1` for the `bias` , because `linear Regression` will deal with the `intercept` itself.
- `degree=2` : means the maximum power for the model: it will compute for you: if you have one `feature` (`x`): `degree=2` creates for us: `x^1` (the original `Feature`), `x^2` (the square of the `feature`). In short the `degree` determines to what degree of the powers we want to go (second, third, ...).
- Then you say `lin_reg.fit(X_poly, y)` : the model learns: `ŷ = θ0 + θ1 x + θ2 x²` .

From your notebook

Polynomial Features with more than one feature

If I have the `features` `a, b` and used `degree=3` : the result will not come out only `a, b, a², b², a³, b³` but it also adds `Combinations` such as `ab, a²b, ab²` . So it can discover relationships **between the features themselves** (something ordinary `linear regression` cannot do).

`Combinatorial Explosion` : whenever the `degree` or the number of `features` increases, the number of new `features` can increase enormously. `Polynomial Regression` may be strong, but a very high `degree` can produce a huge number of `Features` .

Addition

The formula (the number of features after the transformation, including the bias term):

$$\text{number of features} = \frac{(n+d)!}{d!\ n!}$$

Addition

For example: `n=10` and `d=3` gives **286** (one of them is the bias, i.e. 285 actual features). But `n=100` and `d=3` gives about **176 thousand**. And see also the names of the `features` that `PolynomialFeatures(3)` produces for `a` and `b`:

```
PolynomialFeatures(3, include_bias=False).fit(np.zeros((1, 2))).get_feature_names_out(["a", "b"])
```

Output (real run)

```
['a' 'b' 'a^2' 'a b' 'b^2' 'a^3' 'a^2 b' 'a b^2' 'b^3']
```

Practical advice: always use `StandardScaler` after `PolynomialFeatures` (because `x^30` has huge numbers and `scale` is important for training).

9 Overfitting, Underfitting and Learning Curves

From your notebook

What will happen if the degree is very high? Here we enter `Underfitting` and `Overfitting`: the book tried a very large `Polynomial degree`, so the model tries to follow the training data point by point and the curve became very wiggly.

The result: `linear model` ← `Underfitting`. `High-degree model` ← `Overfitting`.

From your notebook

How do I know whether the model underfits or overfits? The book goes back to the idea of `Cross-Validation`:

Case	Training performance	Validation performance
Overfitting	excellent	bad
Underfitting	bad	bad

Addition

This is exactly what we saw in Chapter 2: the `Decision Tree` gave `RMSE = 0` on the `training` (excellent) but about `66,868` in `cross-validation` (bad) ← `Overfitting`. And the `Linear Regression` was bad in both (`68,688` on the `training`) ← `Underfitting` .

Quick treatment: `Underfitting` ← a stronger model / better `features` / reduce the constraints (`regularization`). `Overfitting` ← simplify the model / constrain it (`Regularization`) / get more data.

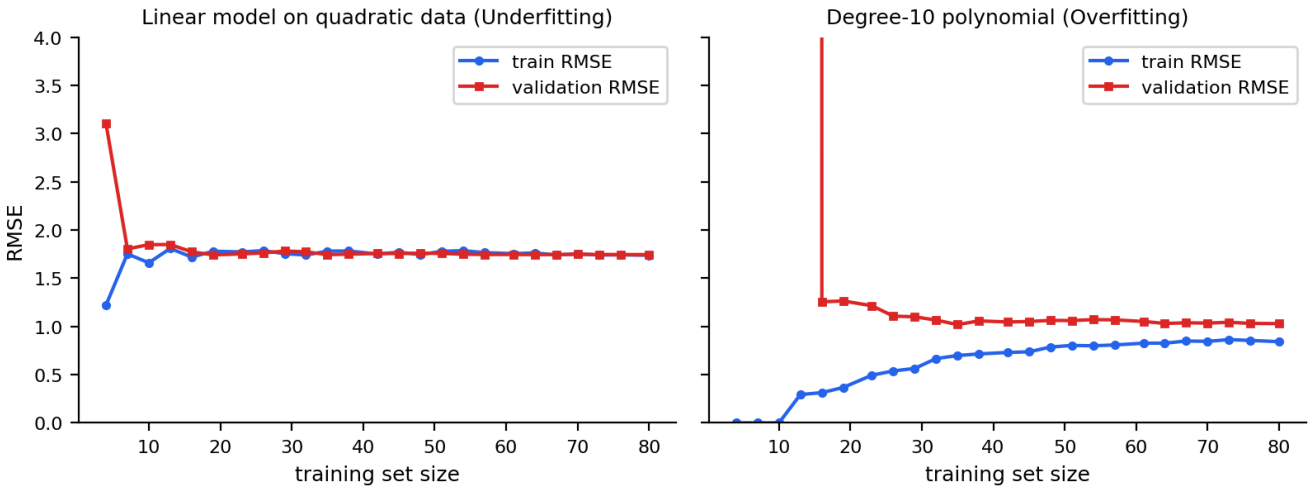
Learning Curves

From your notebook

This is an additional tool to know whether the model `underfits` or `overfits` .

The idea: we follow the `Training error` and the `Validation error` as the size of the `Training set` increases. The book provides `learning_curve()` : and it returns `train_sizes, train_scores, valid_scores` . And we can draw the `errors` against the size of the `training data` .

```
from sklearn.model_selection import learning_curve
train_sizes, train_scores, valid_scores = learning_curve(
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
train_errors = -train_scores.mean(axis=1)    # negate: scores are NEGATIVE rmse
valid_errors = -valid_scores.mean(axis=1)
```



From a real run on quadratic data (synthetic): left: the linear model: the two curves stuck at an RMSE of about 1.74 and there is no benefit from more data (Underfitting). Right: degree 10, the train is about 0.84 and the validation is about 1.03 and the gap narrows with more data (Overfitting).

Addition

Pay attention to the negative: as we explained in Chapter 2 (`cross_val_score`), `scoring="neg_root_mean_squared_error"` returns **negative values**, so we must put a `-` sign in front of it to return the real RMSE.

From your notebook

The `learning curve` for a `linear model` :

If you found:

1. `Training error` starts low then increases and settles.
2. `Validation error` starts high then decreases.
3. In the end the two curves are close to each other and relatively high.

This is a shape pointing to `Underfitting` .

And if you found:

1. `Training error` is very low.
2. `Validation error` is higher.
3. And there is a clear `Gap` between them.

This is one of the signs of `Overfitting` . And the book mentions that increasing the `Training data` may help reduce the `Gap` .

Addition

The rule that will help you in the revision: if the two curves stuck to each other at a high error, then **more data will not help** (the model itself is weak). If there is a gap, then **more data will probably help**.

10 Generalization Error and the Bias/Variance Trade-off

From your notebook

Generalization Error: in short it is the error rate of the model when it works on completely new data it has not seen before (`Unseen Data`). The lower this error the "smarter" the model and the better its ability to generalize.

The total error in any model consists of 3 main things:

From your notebook

① The Bias (bias or "stubbornness")

Meaning: the model is simpler than necessary and has fixed convictions it does not change whatever happens.

Example: like trying to draw a straight line to connect points that are originally on a curve shape (`Quadratic`). The model refuses to understand the curvature and has a "bias" toward the straight line.

The result: `Underfitting` (a shortfall in learning): the model is dumb and does not understand anything, neither in training nor in the (new) test.

From your notebook

② The Variance (variance or "quick influence")

Meaning: the model is more complicated than necessary and very sensitive to any slight change in the training data.

Example: if you brought a `polynomial` of a very high degree (`High-degree Polynomial`) and let it pass through every point with scary precision, if you changed just one point in the training data, the shape of the whole curve will change in a strange way.

The result: `Overfitting` : the model memorized the training data by heart with all the `noise` in it, but when it sees new data it fails completely.

From your notebook

③ The Irreducible Error (the error that cannot be removed)

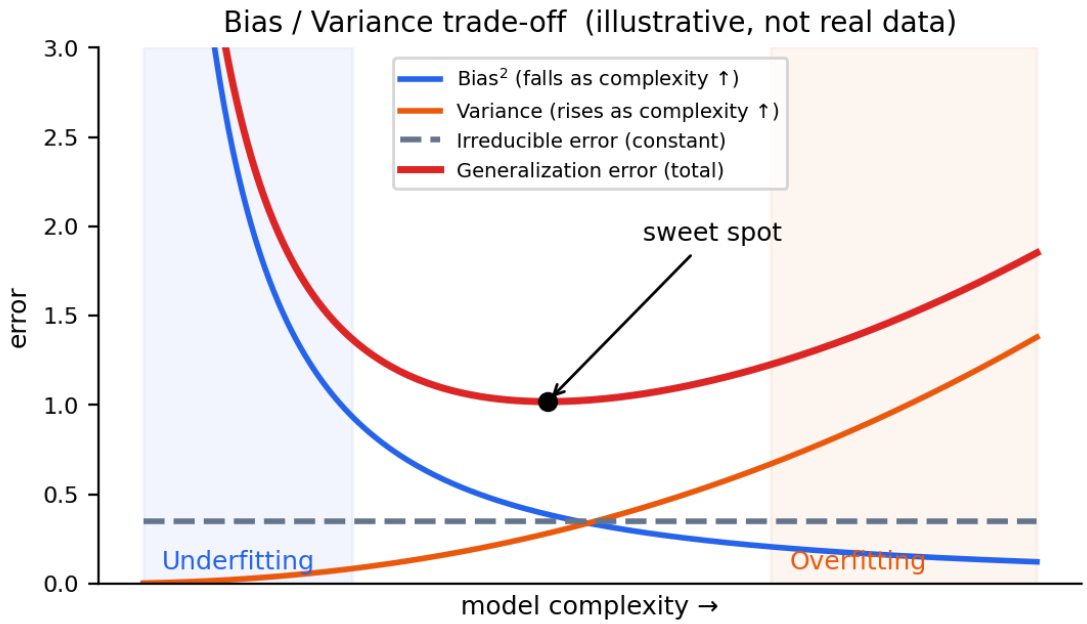
Meaning: this is the remaining error due to the " `Noise` " originally present in life itself and in the way the data was collected (like an error in a measuring device reading, or wrongly entered data from the start).

The solution: however genius the model is it will not be able to solve this; the only solution is that you clean your data (`Data cleaning`) and fix it.

Addition

This is the mathematical expression in the book: the three parts add up. Two of them (Bias and Variance) we can influence by choosing the model, and the third is constant.

$$\text{Generalization Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$



The figure is illustrative: on the left High Bias (Underfitting), on the right High Variance (Overfitting), and the red line (the total error) is at its minimum in the middle.

From your notebook

What is the Bias/Variance Trade-off (the dilemma of push and pull)?

The relationship between `Bias` and `Variance` is like a "see-saw" (`See-saw`): the two cannot both be low at the same time on their own, there must be a balance between them:

- When you **raise** `Model Complexity ↑` : the `Bias` ↓ (the model matches and gets closer to the details of the data) and the `Variance` ↑ (the model becomes sensitive and changes easily).
- When you **lower** `Model Complexity ↓` : the `Variance` ↓ (the model becomes stable and is not affected by every wind) and the `Bias` ↑ (the model becomes very superficial and incapable of understanding things).

The `Trade-off` : your job as an engineer is to stand at the right "point", to reach the lowest possible sum (`Bias + Variance`); then you build the best model that achieves the lowest possible `Generalization error` .

	High Bias	**High Variance**
Model	too simple	too complex
Training error	high	very low
Validation error	high (close to the training)	high (far from the training = a gap)
Case	Underfitting	Overfitting
Treatment	a stronger model or better features	more data or Regularization or a simpler model

11 Regularization and Ridge Regression

From your notebook

Regularization: the idea is that if you have `overfitting` you can constrain the model.

The idea: instead of allowing the model `weights` to take any values, we impose constraints on them so that the model does not become more complex than necessary.

In the `linear model` the book introduces 3 types: ① **Ridge** ② **Lasso** ③ **Elastic Net**

From your notebook

Ridge Regression

Ridge = linear Regression + L2 Regularization. We add a `Penalty` to the `MSE` that depends on the **squares** of the `weights` :

The goal: the model not only reduces the `MSE` error but also tries to make the `weights` **small**.

$$J(\theta) = \text{MSE}(\theta) + \frac{\alpha}{m}\sum_{i=1}^{n}\theta_i^2$$

From your notebook

`alpha` : this `Hyperparameter` controls the strength of the `Regularization` .

- `alpha = 0` ← ordinary `linear Regression` .
- Higher `alpha` ← stronger `Regularization` ← smaller `weights` ← simpler model.

If `alpha` is very large: the `weights` approach zero, and so the prediction becomes closer to a flat line around the mean of the data.

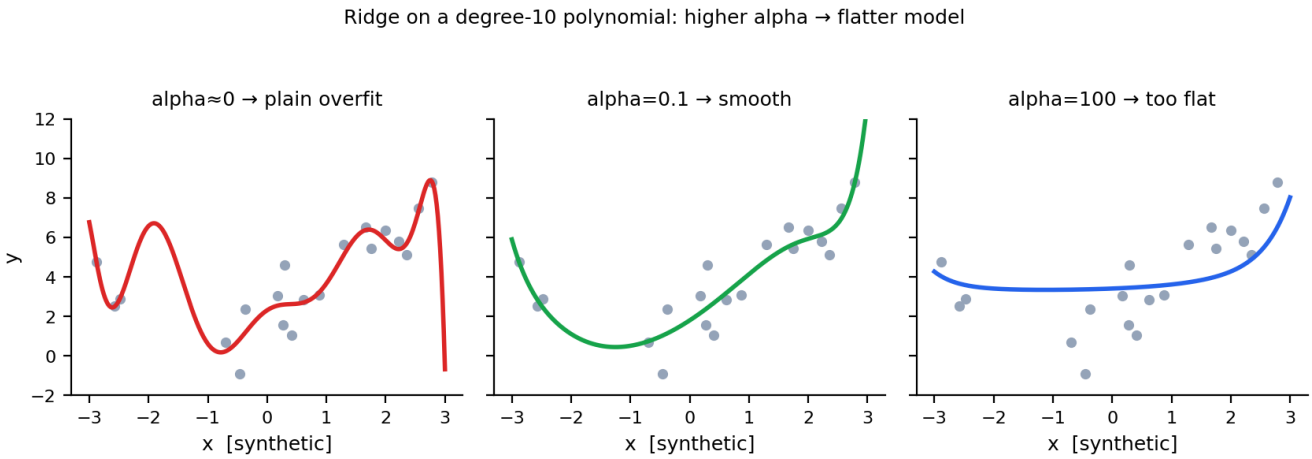
Important: the `Bias / Intercept` (`θ0`) is **not** `Regularized` in `Ridge` (according to the book's formula the sum starts from `i=1`).

From your notebook

Why is Scaling important with Ridge? The `Regularization` depends on the **size** of the `weights` , so a difference in `Feature scales` affects the penalty. That is why the book says: `Scale the data before Ridge regression` (and it is true for most `Regularized models`).

From your notebook

Polynomial Regression with Ridge: we can do: `Polynomial Features` ← `Standard Scaler` ← `Ridge`. And whenever you increase `alpha`: the `weights` ↓, the `Model` becomes `flatter`, the `Bias` ↑ and the `Variance` ↓.



The same pipeline (degree=10 + StandardScaler + Ridge) with 3 alpha values on 20 points (synthetic): alpha≈0 learned the wiggles, alpha=0.1 is reasonable, alpha=100 is too flat (Underfitting).

Ridge in Scikit-learn

```
from sklearn.linear_model import Ridge
ridge_reg = Ridge(alpha=0.1, solver="cholesky")
ridge_reg.fit(X, y)
print(ridge_reg.predict([[1.5]]))

from sklearn.linear_model import RidgeCV
ridge_cv = RidgeCV(alphas=[0.1, 1.0, 10.0])
ridge_cv.fit(X, y.ravel())
print(ridge_cv.alpha_)
```

Output (real run)

```
[8.3659]
```

From your notebook

Here: `Ridge` + `alpha` = `Regularized linear regression`; and `solver=` is just a way to solve the problem mathematically in this example.

- The book says that you can also use `SGDRegressor(penalty="l2")` to get the same type of `L2 Regularization` with the `SGD` style.

From your notebook

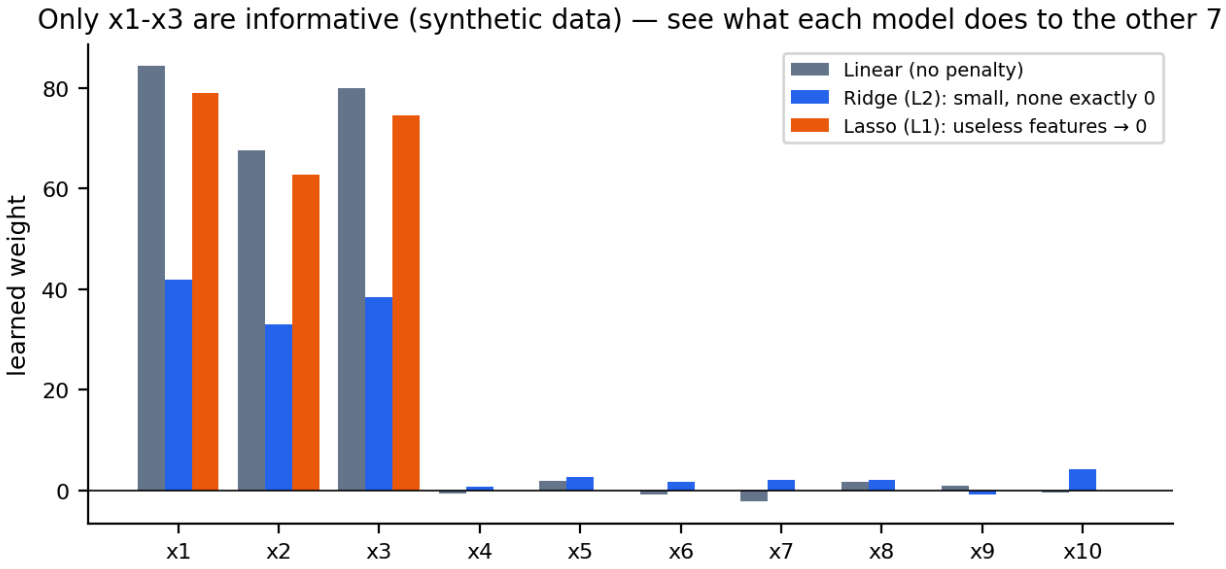
`RidgeCV` : instead of choosing `alpha` manually: Scikit-learn has `RidgeCV` : it does `Ridge Regression` and chooses the `Hyperparameter` using `Cross-Validation`. The book says it is roughly like `GridSearchCV` but specialized and optimized for `Ridge` .

Output (real run)

```
0.1
```

12 Lasso, Elastic Net and Early Stopping

```
from sklearn.linear_model import Lasso
lasso_reg = Lasso(alpha=0.1)
lasso_reg.fit(X, y)
```



From a real run (synthetic): 10 features, only 3 of them useful. Ridge made all the weights smaller but did not zero a single one. Lasso zeroed 7 of them (exactly the 7 not useful) and left the 3 useful.

From your notebook

Lasso Regression

`Lasso` is also a `Regularized linear regression` , but it uses `L1 Regularization` instead of `L2` . That is, the `Penalty` depends on the **absolute values** of the `weights` (not their squares).

$$J(\theta) = \text{MSE}(\theta) + \alpha \sum_{i=1}^{n} |\theta_i|$$

From your notebook

The most important practical feature of Lasso: it can make some `weights` equal exactly **zero**, and so it automatically performs a `Feature selection` process (i.e. the unimportant `Features` may actually disappear from the model).

And this is a very important difference: `Ridge` ← `weights` small. `Lasso` ← some `weights` may become **zero**.

Note / Correction

The Penalty constants: the form of the equations (Ridge / Lasso / Elastic) above is the conceptual formula. The constant that multiplies `alpha` (like `1/m` or `2`) differs from book to book and from library to library, so do not expect the same `alpha` value to give the same result in different tools. The idea itself does not change.

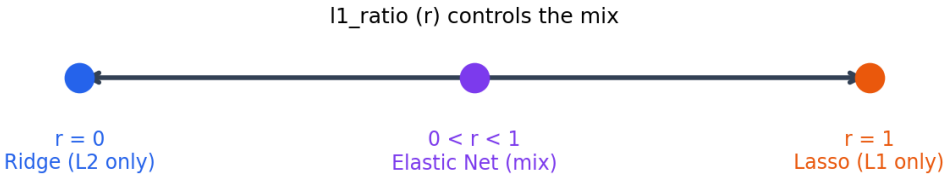
From your notebook

Elastic Net

A middle solution between `Ridge` and `Lasso`, and it uses a mixture of `L1 + L2`. And it has a `Hyperparameter` named `r` (in sklearn its name is `l1_ratio`) that controls the mixing ratio.

- `r = 0` ← `Ridge`.
- `r = 1` ← `Lasso`.
- So `0 < r < 1` is a mixture between them.

$$J(\theta) = \text{MSE}(\theta) + r\alpha \sum_{i=1}^{n} |\theta_i| + (1-r)\alpha' \sum_{i=1}^{n} \theta_i^2$$



Elastic Net: you control it with r: from pure Ridge to pure Lasso.

From your notebook

- `Ridge` ← a good `Default`.
- If you expect only a few `features` to be useful ← `Lasso` / `Elastic Net`.
- `Elastic Net` is often preferred over `Lasso` when the `Features` are very many or there is strong `Correlation` among them (because `Lasso` may behave unstably in some of these cases).

```
from sklearn.linear_model import ElasticNet
elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
elastic_net.fit(X, y)
```

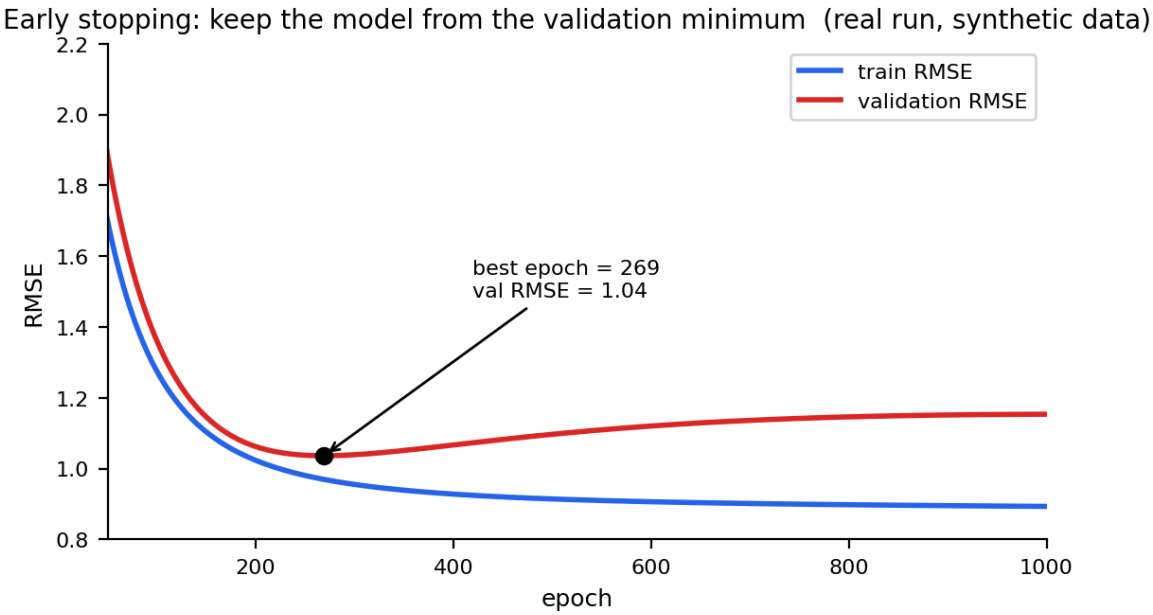
Model	Type	What does it do?	Use it when...
Ridge	L2	shrinks all the weights	the default case, all the features are somewhat useful
Lasso	L1	zeroes the weights of some features	you have many features and few of them are useful
Elastic Net	a mixture of L1 + L2	a mixture of both	many features or correlated with each other

Early Stopping

From your notebook

Another way of `Regularization`, especially with `Iterative Learning` like `Gradient Descent`.

The idea: stop the training when the `Validation error` reaches its minimum point, before it starts to increase (`overfitting`).



From a real run (synthetic, degree=20): train keeps going down, but validation reaches its minimum at epoch about 269 and after that it starts to go up a bit (the start of the overfitting). The difference is small on this data, but the idea is exactly the same.

```
from copy import deepcopy
from sklearn.metrics import root_mean_squared_error

sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
best_valid_rmse = float("inf")
best_epoch = 0
```

```
for epoch in range(1000):
    sgd_reg.partial_fit(X_train_prep, y_train)          # one epoch, incrementally
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = root_mean_squared_error(y_valid, y_valid_predict)
    if val_error < best_valid_rmse:                     # better than any previous result
        best_valid_rmse = val_error
        best_epoch = epoch
        best_model = deepcopy(sgd_reg)                  # keep a copy of the best state
print(best_epoch, best_valid_rmse)
```

From your notebook

That is: `Epochs ↑` ← `Training error ↓` continuously roughly, `Validation error ↓` until the `minimum`, then it starts to increase ← `overfitting ↑`.

Early stopping code

Output (real run)

```
best epoch = 269, best validation RMSE = 1.0365
```

```
from sklearn.base import clone
print(hasattr(clone(sgd_reg), "coef_"))       # does the clone have learned weights?
print(hasattr(deepcopy(sgd_reg), "coef_"))    # does the deepcopy?
```

From your notebook

- `partial_fit()` instead of `fit()` so that the model learns `incrementally` step by step.
- After each `epoch`: `y_valid_predict = sgd_reg.predict(X_valid_prep)`; then we compute `val_error`.
- If the `Validation RMSE` is less than any previous result: `best_valid_rmse = val_error` and `best_model = deepcopy(sgd_reg)`. That is, keep a copy of the best state the model has reached so far.

An important point: in the book's code this does not really stop the training; it only keeps the best model, and after the training ends you can go back to it.

From your notebook

deepcopy vs clone

This is a small point but important if you write code:

- `deepcopy(sgd_reg)` ← copies `Learned Parameters` + `Hyperparameters`.
- `sklearn.base.clone()` ← copies **only** the `Hyperparameters` and not the `Learned parameters`.

Output (real run)

```
False
True
```

Addition

That is, if you did `clone` for the trained model and asked it to `predict` it will refuse (because it returned a "new untrained model"), but `deepcopy` is the one that saves the training state.

And in some versions of the code you may meet `mean_squared_error` instead of `root_mean_squared_error`; the comparison `val_error < best` remains correct (because the root is an increasing function) but the printed numbers will be MSE, not RMSE.

13 Logistic Regression

From your notebook

Here the chapter moves from `Regression` to `Classification`.

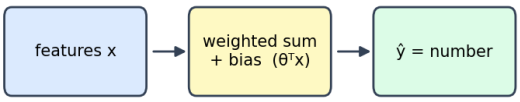
It is used to estimate the **probability** that an `instance` belongs to the `Positive class`. For example `Spam probability = 0.87`.

- If the probability is greater than the `Threshold` ← (usually 50% by default) `1` ← `Positive`.
- Otherwise `0` ← `Negative`.

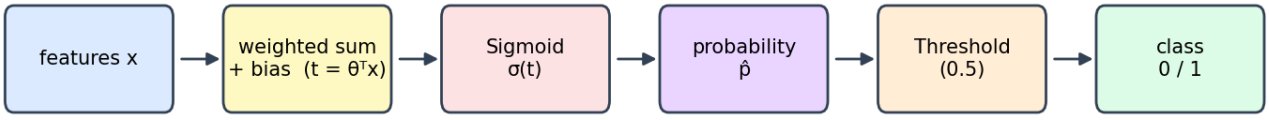
From your notebook

The model first does: `Weighted sum + Bias`, i.e. `t = θ^T x`:

$$t = \theta^T \mathbf{x}$$



Linear Regression: the output of the weighted sum is the number itself.

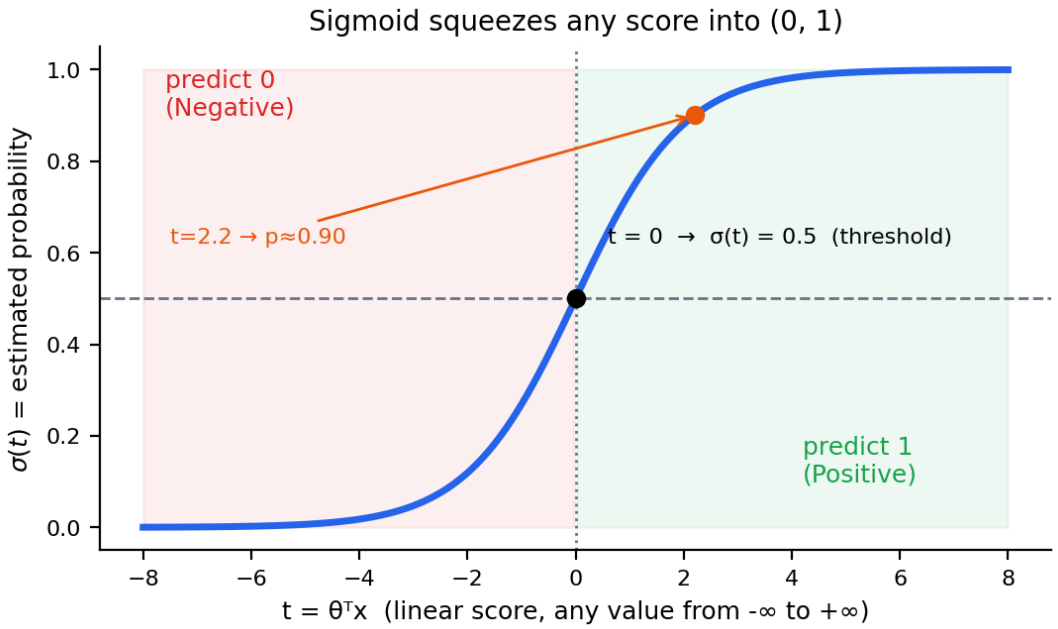


Logistic Regression: the same weighted sum, but after it the Sigmoid and then the Threshold.

From your notebook

But `t` does not come out directly as in `linear regression`; instead it is passed to the `sigmoid / Logistic function`:

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$



The Sigmoid: when t = 0 the probability is exactly 0.5 (and that is the threshold). t positive = Positive, t negative = Negative.

From your notebook

Why Sigmoid? Because it converts any value whatever it is (from `−∞` to `+∞`) into a value between `0` and `1`, so we can consider it an `Estimated Probability` (for example `0.9` = a high probability that it is `Positive`).

Prediction using `Threshold = 0.5`:

$$\hat{y} = 0 \text{ if } \hat{p} < 0.5, \quad 1 \text{ if } \hat{p} \geq 0.5$$

From your notebook

And because `Sigmoid(0) = 0.5`, if `θ^T x > 0` ← `Probability > 0.5` ← `Positive`.

From your notebook

Logit

The book calls the `score` before the `sigmoid` by the name `logit`, and it is connected with `log-odds`, and it is the `inverse` of the `logistic function`:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = t$$

From your notebook

You do not need to drown in the mathematics now, what matters is you know: `linear score` ← `Sigmoid` ← `Probability` ← `Threshold` ← `Prediction`.

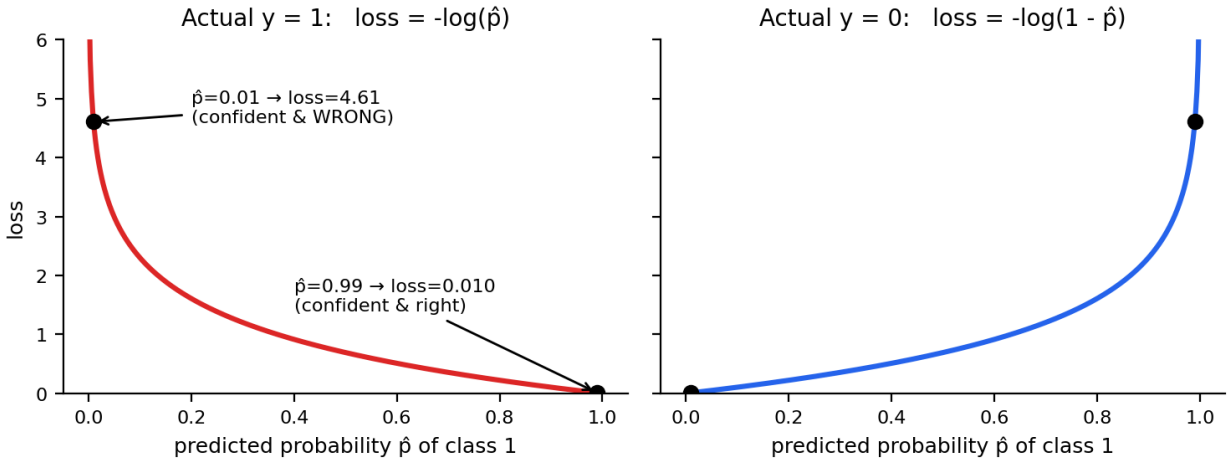
From your notebook

How is Logistic Regression trained?

The training goal: raise the `Probability` for the `Positive instances` and lower the `probability` for the `Negative instances`.

And it uses `Log Loss / Logistic Cost Function`. For one `instance`:

$$c(\theta) = -\log(\hat{p}) \text{ if } y = 1, \quad -\log(1-\hat{p}) \text{ if } y = 0$$



Real values: p̂=0.01 while the right answer is 1 = loss 4.61 (huge). p̂=0.99 = loss 0.010 (almost zero). The curve goes up when the model is sure of a wrong answer.

From your notebook

The important meaning: if the model is very sure of a **wrong** answer, the `loss` will be very big. For example `Actual = 1` and `Prediction probability = 0.01` ← catastrophic error ← `loss` large. But `Actual = 1` and `Prediction probability = 0.99` ← `loss` close to zero.

From your notebook

Log loss over the whole Dataset: we take the **mean** of the `cost` over all the `Training instances`:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{p}^{(i)}\right) + \left(1-y^{(i)}\right)\log\left(1-\hat{p}^{(i)}\right)\right]$$

It is called `log loss` / `Logistic Regression Cost Function`.

Addition

There is no `Normal Equation` for `Logistic Regression` (there is no closed direct solution), so training is with `Gradient Descent` (or improved versions of it). And luckily the `log loss` is also `convex`, so `GD` is guaranteed to reach the global `minimum`.

A real example on Iris: we predict whether the flower is `Iris virginica` or not from the `petal width` only.

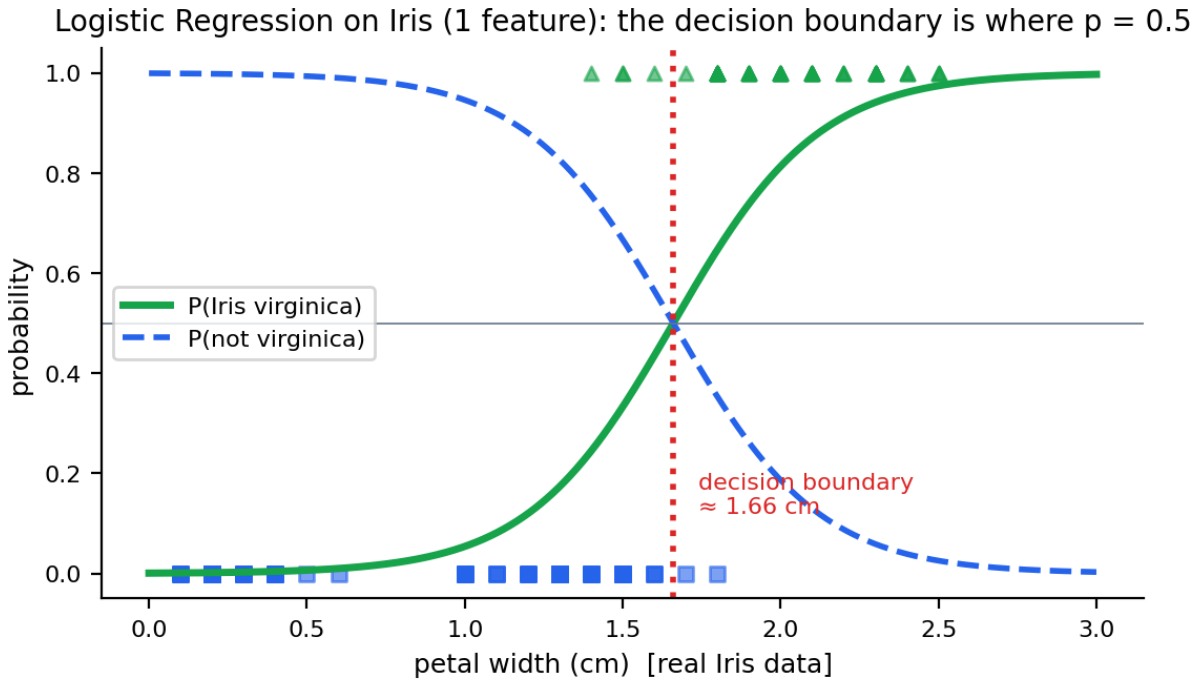
```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

iris = load_iris(as_frame=True)
X = iris.data[["petal width (cm)"]].values
y = (iris.target == 2).astype(int)         # 1 = Iris virginica, 0 = not

log_reg = LogisticRegression(random_state=42)
log_reg.fit(X, y)
print(log_reg.predict_proba([[1.7], [1.5]]))
print(log_reg.predict([[1.7], [1.5]]))
```

Output (real run)

```
[[0.457 0.543]
 [0.667 0.333]]
[1 0]
```



From the real Iris data: the probability crosses 50% at a petal width of about 1.66 cm (the Decision boundary). At 1.7 cm the model says virginica with a probability of 54%, and at 1.5 cm it says no with a probability of 67%.

14 Softmax Regression and Cross Entropy

From your notebook

The difference between it and Linear Regression:

	Linear Regression	Logistic Regression
The steps	Weighted sum → Prediction Value	Weighted sum → Sigmoid → Probability → Class
The output	a real number	a probability between 0 and 1 (then a class)
The Cost	MSE	Log loss

From your notebook

Decision Boundary

It is like a separating line or wall that the model draws to decide based on it: this data belongs to which `class`?

From your notebook

The C in Logistic Regression

An important remark from the book: the `Hyperparameter` for the strength of the `Regularization` in `Scikit-learn` `LogisticRegression` is named `C`, and it is the **inverse** of the strength of the `Regularization`:

- `C ↑` ← `Regularization ↓`
- `C ↓` ← `Regularization ↑`

Addition

That is, the opposite of `alpha` in Ridge/Lasso (roughly `C = 1/alpha`). And the `default` is `C=1.0` with `L2`. If the model `overfits` reduce `C`, and if it `underfits` increase `C`.

From your notebook

The ordinary `Logistic Regression` is used for `Binary classification`; the idea if you have more than one `class`, directly, and this is the `Softmax Regression` / `Multinomial logistic Regression`.

And this is different from the way we met in Ch 3 with `OvR / OvO`: `Softmax` can handle `Multiclass` directly.

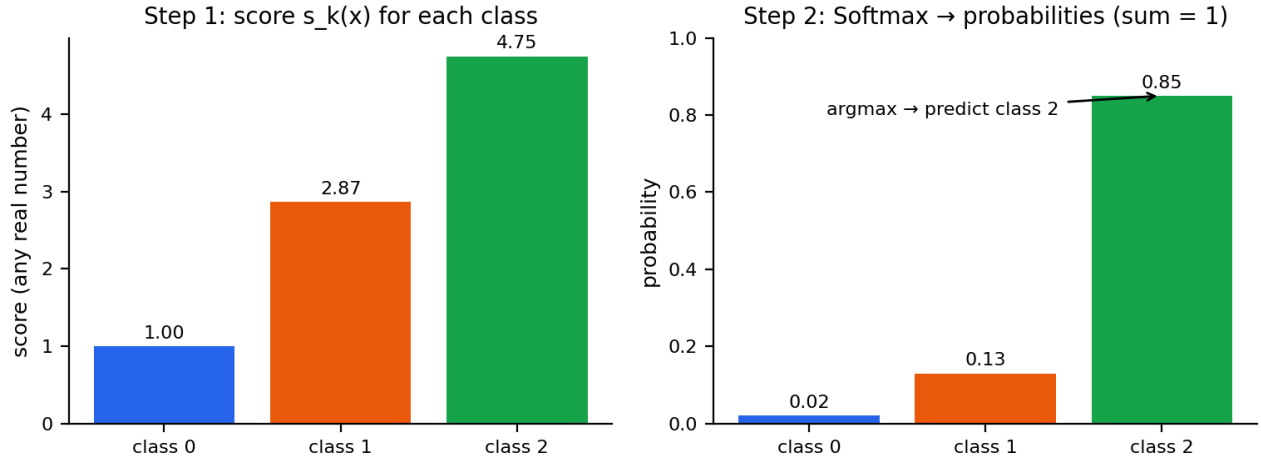
From your notebook

How does Softmax work if we have K classes?

First: it computes a `Score` for each `Class`:

$$s_k(\mathbf{x}) = (\theta^{(k)})^T \mathbf{x}$$

That is, each `Class` has its own `Parameter vector`. For example 3 classes: `class 0 -> score`, `class 1 -> score`, `class 2 -> score`.



The same numbers in your notebook (0.85 / 0.13 / 0.02): left: the raw scores (any number), right: after Softmax they became probabilities summing to 1. The real values (computed by the equation).

From your notebook

After that the `Softmax` converts the `Scores` to `Probabilities`:

$$\hat{p}_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^{K} \exp(s_j(\mathbf{x}))}$$

From your notebook

The idea: it converts all the `scores` into probabilities whose sum = 1. For example: `Class 0 -> 0.02`, `Class 1 -> 0.13`, `Class 2 -> 0.85`. ∴ their sum = 1.

From your notebook

Prediction in Softmax

It chooses the owner of the largest `Probability`:

$$\hat{y} = \text{argmax}_k \, \hat{p}_k$$

That is, the `Class` with the largest `Score / Probability` is the `prediction`.

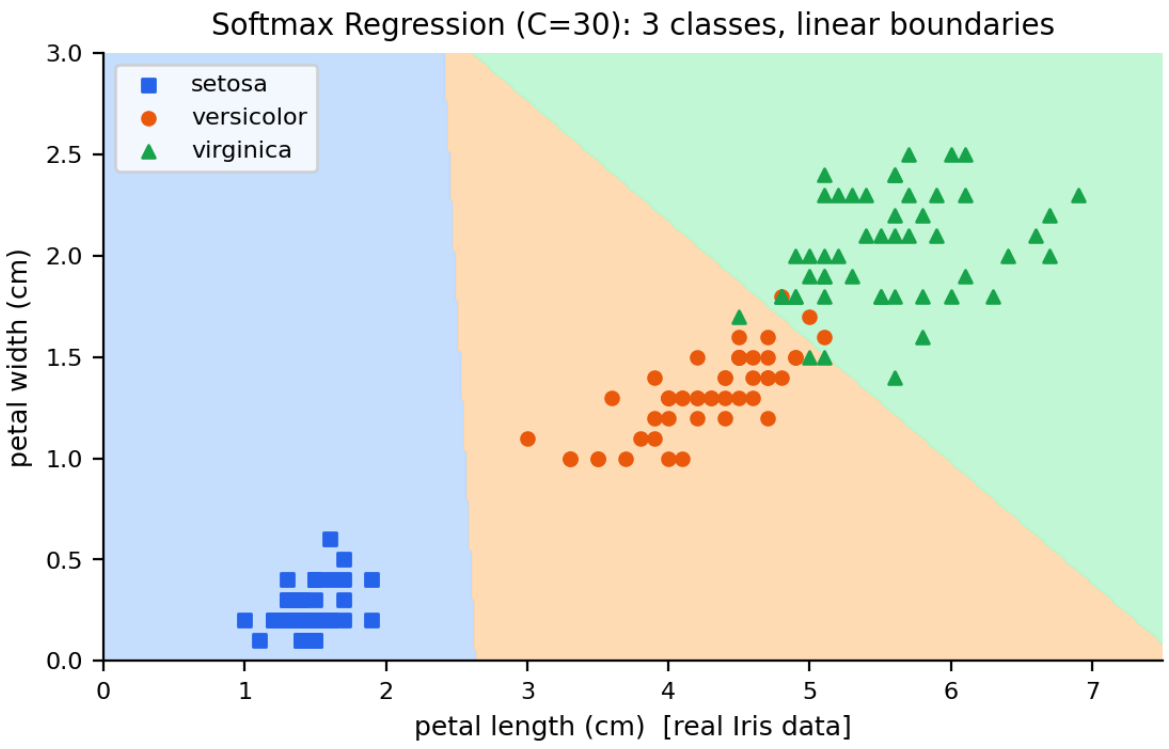
From your notebook

Softmax with Multioutput

The book mentions that `Softmax` is `Multiclass`: it chooses only **one** `class`, and not `Multilabel / Multioutput`.

```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris.target                          # 3 classes: 0, 1, 2

softmax_reg = LogisticRegression(C=30, random_state=42)    # softmax by default for 3+ classes
softmax_reg.fit(X, y)
print(softmax_reg.predict([[5, 2]]))
print(softmax_reg.predict_proba([[5, 2]]).round(3))
```



From the real Iris data: Softmax divides the plane into 3 regions with linear boundaries. The point (5, 2) fell in the virginica region with a probability of 97%.

Output (real run)

```
[2]
[[0.000 0.029 0.971]]
```

Note / Correction

In old versions they used to write `multi_class="multinomial"` to make it use Softmax. This `parameter` was deprecated in newer versions (from roughly 1.5), and in the version this file was tried on (scikit-learn 1.8) it was removed completely and writing it gives a `TypeError`. `LogisticRegression` now uses Softmax automatically when there are 3 classes or more, so if your old code has this `parameter`, remove it.

From your notebook

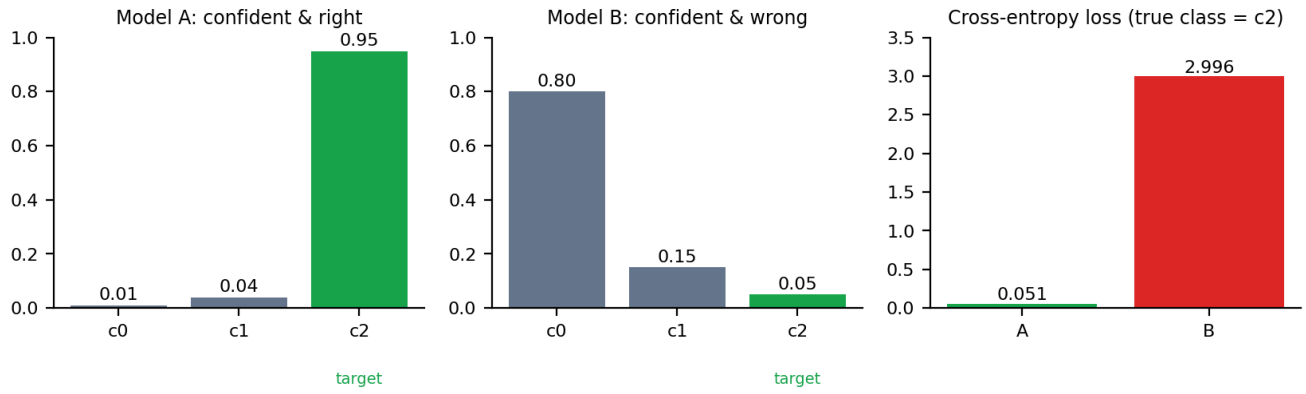
How do we train Softmax?

We want the model to give a high `Probability` for the `Target class`. So it uses the `Cross Entropy loss`:

$$J(\Theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$$

From your notebook

In the case of one correct class: `Target: [0, 0, 1]`. If the model said `[0.01, 0.04, 0.95]`, that is excellent. But if it said `[0.80, 0.15, 0.05]` then `Cross entropy` penalizes it severely because it gave a small `Probability` to the `Target class`.



The real values of loss = −log (the probability of the correct class): Model A has a loss of 0.051, and Model B has a loss of 2.996 (about 58 times).

Addition

If `K = 2` (class one correct and one wrong) the `Cross Entropy` turns exactly into the `log loss` we saw in the `Logistic Regression`. That is, the `Log loss` is a special case of it.

An idea that ties the whole chapter together: `Linear ← MSE`. `Logistic ← Log loss`. `Softmax ← Cross entropy`. And all of them are trained with `Gradient Descent`, and all of them work with `Regularization`.

15 Quick review questions

Answer first in your head, then check the answers on the next page.

1. What is the difference between Parameters and Hyperparameters? And who determines each?
2. What is the Cost Function? And why can the Training loss differ from the Evaluation metric?
3. Write the Normal Equation and say its meaning in simple words.
4. Why does `LinearRegression` in sklearn use the pseudoinverse instead of the direct `inv`?
5. Why don't we always use the Normal Equation? And what slows it down?
6. Explain the idea of Gradient Descent with the mountain and fog analogy, and what is the Gradient?
7. What happens if the learning rate is very small and very large?
8. Why is Scaling important with Gradient Descent? And does the Normal Equation need scaling?
9. Why is the MSE in Linear Regression being convex useful?
10. What is the difference between Batch, Stochastic and Mini-batch GD? And what is the advantage and disadvantage of each?
11. What does epoch mean? And how many updates in 50 epochs if m=100 in SGD?
12. What is the learning schedule? And why does SGD need it?
13. Why must we shuffle the data with SGD? (iid)
14. How do we do Polynomial Regression with a linear model? And what is the role of `include_bias=False`?
15. What is the Combinatorial Explosion in PolynomialFeatures?
16. How do you distinguish between Overfitting and Underfitting from the Learning Curves? And what is the difference in the treatment?
17. Explain Bias, Variance, Irreducible error, and what is the Trade-off?
18. What is the difference between Ridge and Lasso? And when do you use Elastic Net?
19. Why must we do scaling before Ridge? And is the bias θ0 regularized?
20. How does Early Stopping work? And what is the difference between `deepcopy` and `clone`?
21. How does Logistic Regression convert the score to a class? (sigmoid + threshold)
22. Why does the log loss penalize a model that is confident and wrong severely?
23. What is the meaning of `C` in `LogisticRegression`? And its relation to regularization?
24. What is the difference between Logistic Regression and Softmax Regression? And what is the Cross Entropy?

16 Review question answers

1. **Parameters:** the model learns them from the data (θ0, θ1, ...). **Hyperparameters:** we determine them before training (η, epochs, alpha, degree, C).
2. A function that takes θ and returns a number that measures the model's badness (like MSE). The training loss may be easier to optimize or have extra parts (regularization), but the final metric may be something else (like Precision/Recall).
3. $\hat{\theta} = (X^T X)^{-1} X^T y$. It computes the best θ (the one that minimizes the MSE) directly from the data without iterations.
4. Because $X^T X$ may not be invertible (duplicated or perfectly correlated features). The pseudoinverse (SVD) works even then, and is more stable.
5. Because it needs $(X^T X)^{-1}$ and its cost grows quickly with the number of features n (about $O(n^{2.4})$ to $O(n^3)$). If n is very large it is slow.
6. You are on top of a mountain in fog, you do not see the valley but you feel the slope of the ground, so you go down in the direction of the descent. The Gradient = the set of partial derivatives of the cost with respect to each θ, it gives the direction of the error increase, so we move opposite to it.
7. Very small: it arrives but very slowly. Very large: it may jump over the minimum and keep jumping or diverge.
8. Features of different sizes make the cost surface a long thin valley, so GD crawls. The Normal Equation does not need scaling.
9. Because it is a bowl: one global minimum and no local minima, so GD is guaranteed to approach it (if η is suitable and we continue enough).
10. Batch: all the data in every step (stable but slow). SGD: one sample in every step (very fast, but cannot settle, noisy). Mini-batch: a small group (balance + benefits from the GPU).
11. epoch = one complete pass over the training set. In SGD every epoch has m steps, so 50 epochs × 100 = 5000 updates.
12. A function that determines how the learning rate decreases during training (for example t0/(t+t1)). SGD needs it because it keeps jumping around the minimum; we start with big steps and then make them smaller.
13. If the data is ordered (all A then all B) the updates are affected by the order and the training path becomes unbalanced. The shuffle brings the samples closer to iid (independent and from the same distribution).
14. We add new features (x^2, x^3, ...) with PolynomialFeatures and then use an ordinary LinearRegression. include_bias=False because LinearRegression adds the intercept itself.
15. The number of new features increases enormously with the degree and the number of original features ($(n+d)!/(d!n!)$), because of the powers and the combinations between them.
16. Underfitting: the two curves are close and high (more data will not help; it needs a stronger model). Overfitting: the train is very low and the validation is higher with a gap (more data, regularization or a simpler model help).

17. Bias: excessive simplicity (Underfitting). Variance: excessive sensitivity to the data (Overfitting). Irreducible: the noise in the data itself. The Trade-off: higher complexity reduces the bias and increases the variance, so we choose the balance point.
18. Ridge (L2) shrinks all the weights. Lasso (L1) zeroes some of them (feature selection). Elastic Net is a mixture of them (r); we prefer it when the features are many or correlated with each other.
19. Because the penalty depends on the size of the weights, and the difference in scales affects it. And the bias θ0 is not regularized.
20. We stop (or keep the best copy) at the lowest validation error before it starts to go up. deepcopy copies the learned parameters + hyperparameters, and clone copies the hyperparameters only.
21. It computes t = θ^T x, then sigmoid(t) produces a probability between 0 and 1, and then the threshold (0.5): if ≥ 0.5 it is class 1, otherwise 0.
22. Because −log(p̂) goes to infinity when p̂ approaches 0 and the right answer is 1. For example p̂=0.01 gives a loss of 4.6 versus 0.01 when p̂=0.99.
23. C is the inverse of the strength of the regularization: a large C = weaker regularization, a small C = stronger. (Roughly C = 1/alpha.)
24. Softmax handles more than one class directly (probabilities summing to 1, with a parameter vector for each class), and the ordinary Logistic is binary. Cross Entropy is its loss: −(1/m) ΣΣ y log(p̂), and it penalizes when the correct class gets a small probability. If K=2 it becomes the log loss.